

Rapport J06 - Consolidation SQL, transactions, IDENTITY et séquences PostgreSQL

Plan d'exécution SQL/Data - Semaine 01

1. Identification de la séance

- Parcours : Plan d'exécution SQL/Data - 16 semaines
- Semaine : 01
- Jour : J06
- Base de données : sql_training
- Table de travail : raw.training_events
- Environnement : Windows 11, PowerShell, PostgreSQL 17.5, psql, VS Code, Git/GitHub
- Durée prévue : 75-90 min
- Durée réelle : 89 min
- Écart prévu/réel : -1 min par rapport au plafond de 90 min (séance terminée dans la fenêtre prévue de 75-90 min)

2. Objectifs

J06 est une séance de consolidation autonome des acquis de la semaine. Les objectifs étaient de :

1. relire et filtrer les données de raw.training_events;
2. utiliser WHERE, AND, OR, les comparateurs et IS NOT NULL;
3. consolider le tri avec ORDER BY;
4. exécuter une transaction contrôlée avec BEGIN et ROLLBACK;
5. observer le comportement d'une colonne IDENTITY et de sa séquence PostgreSQL;
6. transformer les exercices exécutés en un script SQL reproductible;
7. versionner les preuves et le script dans Git.

3. État initial

La connexion a été effectuée sur la base sql_training avec l'utilisateur postgres. L'audit initial a confirmé le schéma courant public et 7 lignes dans raw.training_events.

```
SELECT
    current_database() AS database_name,
    current_user AS user_name,
    current_schema() AS current_schema;
```

```
SELECT COUNT(*) AS total_events
FROM raw.training_events;
```

```
SELECT
    event_id,
    event_type,
    source_system,
    event_value,
    event_timestamp
FROM raw.training_events
ORDER BY event_id;
```

État observé : 7 lignes, avec les identifiants 1, 3, 4, 5, 6, 7 et 8.

4. Challenge autonome de filtrage

A - sensor_gateway

```
SELECT
    event_id,
    event_type,
    source_system,
```

```

        event_value
FROM raw.training_events
WHERE source_system = 'sensor_gateway';

```

Résultat : 2 lignes.

B - event_value > 100

```

SELECT
    event_id,
    event_type,
    event_value
FROM raw.training_events
WHERE event_value > 100
ORDER BY event_value DESC;

```

Résultat : 3 lignes.

C - microgrid_poc et event_value >= 200

```

SELECT
    event_id,
    event_type,
    source_system,
    event_value
FROM raw.training_events
WHERE source_system = 'microgrid_poc'
    AND event_value >= 200
ORDER BY event_value DESC;

```

Résultat : 2 lignes.

D - temperature ou voltage

```

SELECT
    event_id,
    event_type,
    source_system,
    event_value
FROM raw.training_events
WHERE event_type = 'temperature'
    OR event_type = 'voltage'
ORDER BY event_type DESC, event_value DESC;

```

Résultat : 3 lignes.

E - valeurs non NULL

```

SELECT
    event_id,
    event_type,
    event_value
FROM raw.training_events
WHERE event_value IS NOT NULL
ORDER BY event_id;

SELECT COUNT(*) AS nombre_non_null
FROM raw.training_events
WHERE event_value IS NOT NULL;

```

Résultat : 7 lignes non NULL.

5. Transaction contrôlée

Une ligne temporaire de type pressure a été insérée à l'intérieur d'une transaction.

```

BEGIN;

SELECT COUNT(*)
FROM raw.training_events;

INSERT INTO raw.training_events (
    event_type,
    source_system,
    event_value,
    event_timestamp
)
VALUES (
    'pressure',
    'sensor_gateway',
    101.35,
    CURRENT_TIMESTAMP
);

```

```
SELECT COUNT(*)
FROM raw.training_events;

ROLLBACK;
```

Résultats :

- avant l'INSERT : 7 lignes;
- pendant la transaction : 8 lignes;
- après ROLLBACK : retour à 7 lignes;
- la ligne pressure n'existe plus.

Validation :

```
SELECT
    event_id,
    event_type
FROM raw.training_events
WHERE event_type = 'pressure';

SELECT COUNT(*)
FROM raw.training_events;
```

6. IDENTITY et séquence PostgreSQL

La séquence associée à event_id a été identifiée avec :

```
SELECT
    pg_get_serial_sequence(
        'raw.training_events',
        'event_id'
    ) AS identity_sequence;
```

Résultat :

raw.training_events_event_id_seq

L'état de la séquence a ensuite été contrôlé :

```
SELECT
    last_value AS sequence_last_value,
    is_called
FROM raw.training_events_event_id_seq;
```

Après la première expérience, la séquence avait atteint 9 alors que la table contenait toujours 7 lignes et que MAX(event_id) valait 8.

Le script complet a ensuite été réexécuté pour validation. La nouvelle insertion transactionnelle annulée a consommé une valeur supplémentaire : la séquence a atteint 10, tandis que la table est restée à 7 lignes avec MAX(event_id) = 8.

```
SELECT
    COUNT(*) AS total_rows,
    MAX(event_id) AS max_event_id
FROM raw.training_events;

SELECT
    last_value AS sequence_last_value
FROM raw.training_events_event_id_seq;
```

État final observé :

Interprétation

ROLLBACK annule les modifications transactionnelles apportées à la table, mais ne remet pas automatiquement en arrière les valeurs déjà consommées par une séquence PostgreSQL. Une colonne IDENTITY garantit la génération d'identifiants, mais ne garantit donc pas une suite continue sans trous.

Conséquence Data Engineering : MAX(id) ne doit pas être interprété comme un nombre de lignes et une séquence ne doit pas être utilisée comme preuve d'absence de suppression, de rollback ou d'échec d'ingestion.

7. Script reproductible

Le travail a été consolidé dans :

documentation/semaine-01/jour-06/code/J06_sql_consolidation_challenge.sql

Le script a été exécuté depuis PowerShell avec `psql -f` et les résultats ont confirmé les exercices, la transaction et le comportement de la séquence.

8. Erreurs, difficultés et résolutions

Saisie multi-ligne dans psql

Une difficulté de saisie a été rencontrée lors de la construction d'une requête multi-ligne. La séance a permis de rappeler qu'une requête non terminée peut être annulée puis ressaisie proprement, plutôt que d'essayer de modifier une ligne déjà validée dans le tampon interactif.

NULL et logique SQL

La distinction suivante a été consolidée :

```
event_value IS NULL
```

teste explicitement l'absence de valeur.

En revanche :

```
event_value = NULL
```

ne produit pas une comparaison vraie : le résultat logique est UNKNOWN. Les tests sur NULL doivent utiliser `IS NULL` ou `IS NOT NULL`.

Séquence après ROLLBACK

Le comportement de la séquence n'est pas une anomalie. La transaction a été annulée, mais l'identifiant demandé à la séquence avait déjà été consommé. La réexécution du script a confirmé le phénomène en faisant passer `last_value` de 9 à 10 sans ajouter de ligne persistante.

9. Captures réalisées

- J06-01_environment-and-data-audit.png
- J06-02_autonomous-select-filtering-challenge-1.png
- J06-02_autonomous-select-filtering-challenge-2.png
- J06-03_transaction-insert-before-rollback.png
- J06-04_transaction-state-after-rollback.png
- J06-05_identity-sequence-identification.png
- J06-06_identity-sequence-after-rollback.png
- J06-07_count-max-id-and-sequence-comparison.png
- J06-08_sql-consolidation-script-vscode.png
- J06-09_sql-script-execution-and-validation-1.png
- J06-09_sql-script-execution-and-validation-2.png
- J06-09_sql-script-execution-and-validation-3.png

10. Git et versionnement

Le dossier J06 a été ajouté au staging :

```
git add .\documentation\semaine-01\jour-06\  
git status --short
```

Le contrôle montre les captures et le script avec le statut A, c'est-à-dire Added / staged. Au moment de génération de ce rapport, le commit/push final J06 reste à effectuer après ajout des rapports eux-mêmes.

Commandes de clôture recommandées :

```
git add .\documentation\semaine-01\jour-06\  
git status --short  
git diff --cached --stat  
git commit -m "docs: complete J06 SQL consolidation and sequence analysis"  
git push  
git status
```

11. Validation des acquis

Acquis validés :

- SELECT et projection de colonnes;
- filtres WHERE;
- combinaisons AND / OR;
- comparateurs numériques;
- ORDER BY;
- IS NULL / IS NOT NULL;
- COUNT(*), MIN, MAX;
- transaction BEGIN / ROLLBACK;
- comportement d'une colonne IDENTITY;
- identification et inspection d'une séquence PostgreSQL;
- exécution reproductible d'un script avec `psql -f`;
- staging Git des preuves et du code.

12. KPI de progression

13. Challenge conceptuel - réponse

Les valeurs suivantes peuvent être simultanément vraies :

- COUNT(*) = 7 parce que sept lignes existent réellement;
- MAX(event_id) = 8 parce que les identifiants ne sont pas nécessairement continus;
- last_value = 10 parce que des valeurs 9 et 10 ont été consommées par des insertions transactionnelles ensuite annulées.

Cela illustre la différence entre état transactionnel des données et génération d'identifiants par séquence.

14. Potentiel de réutilisation pédagogique

J06 peut être réutilisé sous plusieurs formats :

1. Tutoriel court (5-8 min) : « Pourquoi PostgreSQL saute certains ID ? »
2. Vidéo pratique (8-12 min) : démonstration BEGIN -> INSERT -> ROLLBACK -> séquence.
3. Module de formation (30-45 min) : filtres SQL, NULL, transactions et séquences.
4. Article technique / portfolio : expliquer pourquoi COUNT(*), MAX(id) et last_value mesurent trois réalités différentes.
5. Exercice stagiaire : prédire l'état de la table et de la séquence avant d'exécuter une transaction.

15. Conclusion

J06 consolide les fondamentaux SQL de la première semaine et introduit un comportement PostgreSQL important pour la suite du parcours Data Engineering. La séance ne s'est pas limitée à l'écriture de requêtes : les résultats ont été observés, expliqués, transformés en script reproductible et préparés pour le versionnement Git.

L'expérience BEGIN -> INSERT -> ROLLBACK constitue le résultat technique central : une transaction peut restaurer l'état logique de la table sans restaurer l'état de la séquence.

La séance a été clôturée à 89 minutes, soit 1 minute avant la limite de 90 minutes : l'objectif de durée a donc été respecté.

16. Prochaine étape

Avant J07 :

1. ajouter les trois rapports J06 (.md, .docx, .pdf) au dossier jour-06;
2. effectuer le commit et le push J06;
3. vérifier git status;
4. confirmer working tree clean.

J07 pourra ensuite être consacré à la revue et à la consolidation de la Semaine 01.

Annexe - État Git du dossier J06

```
(base) PS D:\2033\SQL - TRAINING> git status --short
?? documentation/semaine-01/jour-06/
(base) PS D:\2033\SQL - TRAINING> git add .\documentation\semaine-01\jour-06\
(base) PS D:\2033\SQL - TRAINING> git status --short
A  documentation/semaine-01/jour-06/captures/J06-01_environment-and-data-audit.png
A  documentation/semaine-01/jour-06/captures/J06-02_autonomous-select-filtering-challenge-1.png
A  documentation/semaine-01/jour-06/captures/J06-02_autonomous-select-filtering-challenge-2.png
A  documentation/semaine-01/jour-06/captures/J06-03_transaction-insert-before-rollback.png
A  documentation/semaine-01/jour-06/captures/J06-04_transaction-state-after-rollback.png
A  documentation/semaine-01/jour-06/captures/J06-05_identity-sequence-identification.png
A  documentation/semaine-01/jour-06/captures/J06-06_identity-sequence-after-rollback.png
A  documentation/semaine-01/jour-06/captures/J06-07_count-max-id-and-sequence-comparison.png
A  documentation/semaine-01/jour-06/captures/J06-08_sql-consolidation-script-vscode.png
A  documentation/semaine-01/jour-06/captures/J06-09_sql-script-execution-and-validation-1.png
A  documentation/semaine-01/jour-06/captures/J06-09_sql-script-execution-and-validation-2.png
A  documentation/semaine-01/jour-06/captures/J06-09_sql-script-execution-and-validation-3.png
A  documentation/semaine-01/jour-06/code/J06_sql_consolidation_challenge.sql
```